MTA

AUDIO EDITOR

Administrator Manual

Deployment, security, storage and operations



Version: 0.2.0 · Revision: 159

Build: 20261006021358

Alessandro De Salvo

braket71@gmail.com

https://github.com/desalvo/mta-audio-editor

License: EUPL-1.2

Digital Audio Workstation with MTA support

Windows macOS Linux Android iOS

Operational guide for deployment, security, storage, backups, desktop/mobile clients, audio processing, MTA format, observability and releases.

Contents

1. Architecture
2. Deployment
3. Configuration
4. Storage and backups
5. Security and users
6. Audio pipeline
7. Desktop and mobile clients
8. MTA format
9. Operations and observability
10. Upgrade and rollback
11. CI/CD and releases
12. Administrator troubleshooting

1. Architecture

MTA Audio Editor uses a FastAPI backend and a web frontend shared by browsers, desktop clients and mobile apps. Rendering, waveform processing, Demucs and MTA generation run on the backend.

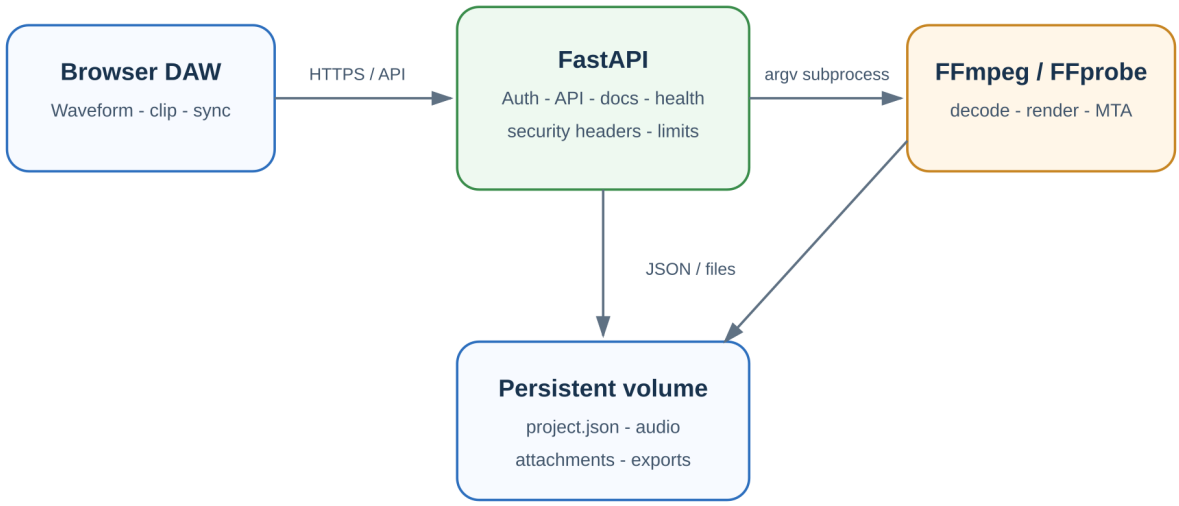


Figure 1 - Clients, API, processing and persistent storage.

Component	Platforms
Server	Linux, Docker, Kubernetes
Desktop	Windows, macOS Intel, macOS Apple Silicon
Mobile	Android, iOS/iPadOS
Browser	Modern browsers with Web Audio API

2. Deployment

Docker Compose should use a persistent volume for `/data/projects`. Kubernetes uses the supplied PVC and an optional Ingress. Application and Ingress upload limits must remain consistent.

```
docker compose up -d
# Kubernetes
kubectl apply -k k8s/overlays/nginx
```

TLS: production deployments should terminate HTTPS at a trusted reverse proxy or Ingress. Configure a stable public URL when email links or OAuth callbacks are enabled.

3. Configuration

Primary settings are supplied through environment variables and Secrets. Administrators should manage bootstrap credentials, upload size, SMTP, OAuth, session security and storage independently.

Area	Examples	Notes
Bootstrap	MTA_ADMIN_USERNAME, MTA_ADMIN_PASSWORD, MTA_ADMIN_EMAIL	Create the initial administrator
Upload	MTA_MAX_UPLOAD_MB	Keep proxy and application limits aligned
Public URL	MTA_PUBLIC_URL	Recommended behind proxies for stable callbacks/ links
SMTP	host, port, TLS, credentials	Store credentials in Secrets

4. Storage and backups

The persistent root contains authentication/configuration data and project directories. Each project can contain its model, audio, attachments, originals, caches and analysis outputs. A full recovery backup must include the complete persistent root.

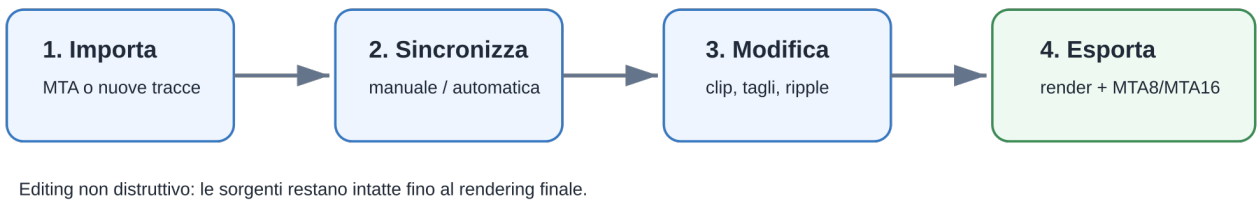
Recommended strategy: use consistent volume snapshots, keep multiple generations and test restores regularly. On Kubernetes verify that the StorageClass/ CSI implementation supports the UID/GID and fsGroup used by the pod.

5. Security and users

Server deployments support local accounts, email verification, administrator approval, optional TOTP and optional OAuth providers. Social providers are disabled by default. OAuth requests are restricted to HTTPS and approved provider hosts.

Native single-user desktop mode bypasses the account database and login screens while retaining the internal authorization context required by local API paths. Secrets and signing material must never be baked into container images.

6. Audio pipeline



Editing non distruttivo: le sorgenti restano intatte fino al rendering finale.

Figure 2 - Non-destructive editing and rendering flow.

FFmpeg and Demucs operations can be CPU/GPU intensive. Track previews are cached with signatures derived from source and relevant processing state. Long-running jobs expose progress and final artifacts through the API.

6.1 Dynamic preview and Render

Dynamic preview buffers track sources on the client and keeps them aligned to a reference clock. Render produces a server-side master. With RealTime meters enabled, track monitor branches feed analysers without becoming audible.

7. Desktop and mobile clients

Desktop clients include a local backend and native file pickers. Mobile apps connect to a remote backend.

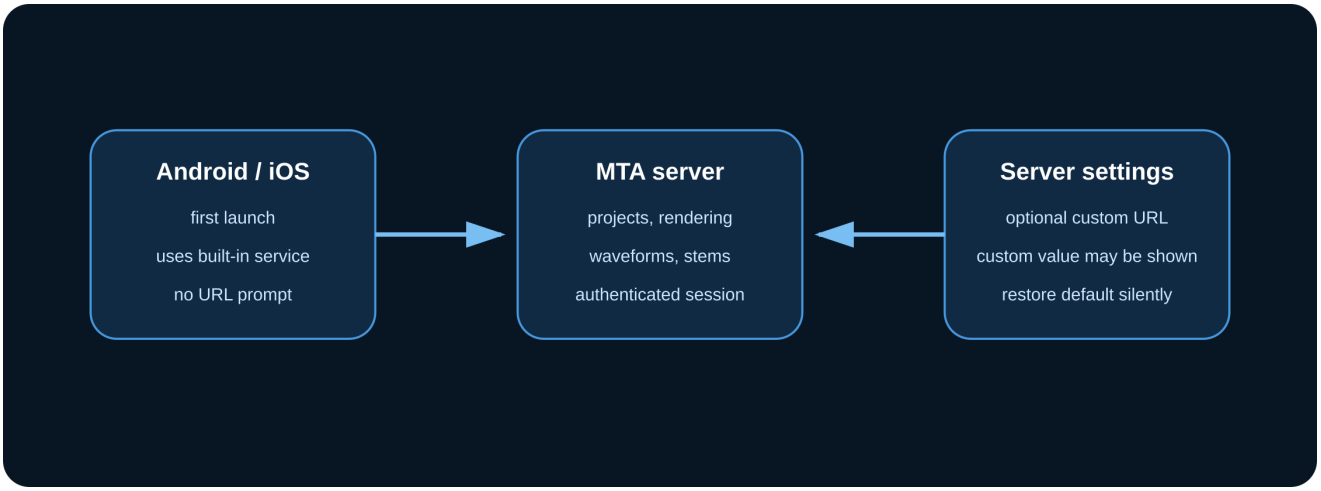


Figure 3 - Mobile connection and custom server settings.

Android and iOS automatically use the preconfigured service without prompting during installation or first launch. The built-in default address remains internal and is never displayed. A custom URL can be configured later and may be shown because it is explicitly user-selected.

8. MTA format

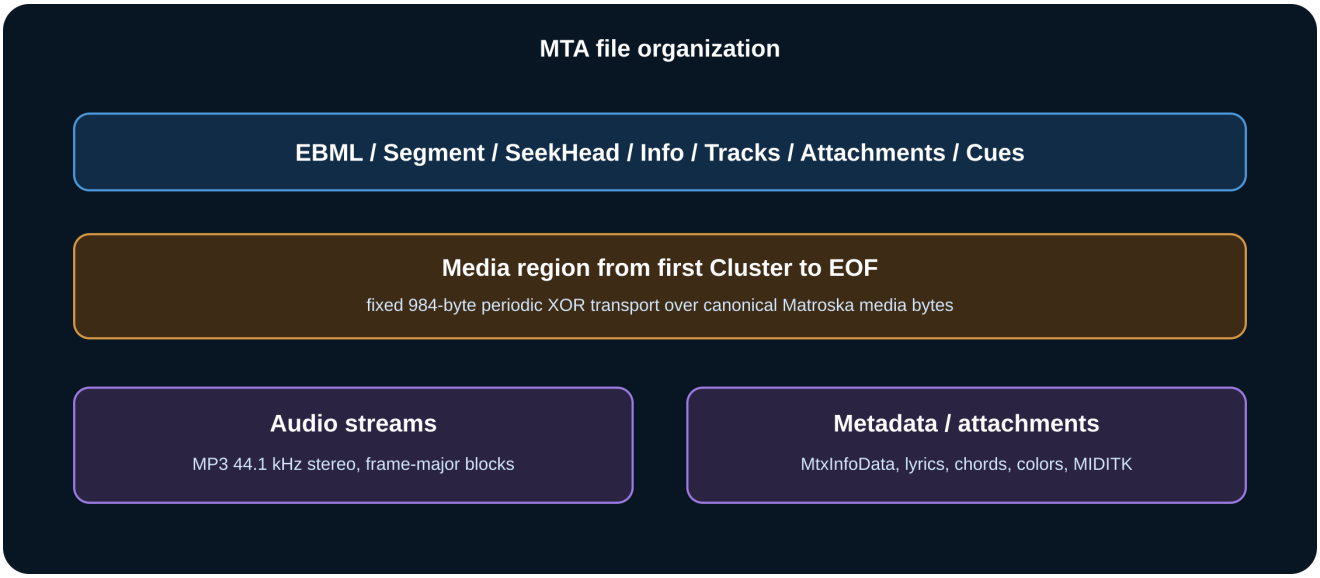


Figure 4 - High-level MTA container organization.

MTA uses EBML/Matroska structures, MP3 audio streams and metadata attachments. The media region starting at the first Cluster uses the documented periodic XOR transport. The complete bilingual specification is stored in `docs/MTA_FORMAT_FINAL_SPEC_IT.md` and `docs/MTA_FORMAT_FINAL_SPEC_EN.md`.

8.1 Click/Melody profiles

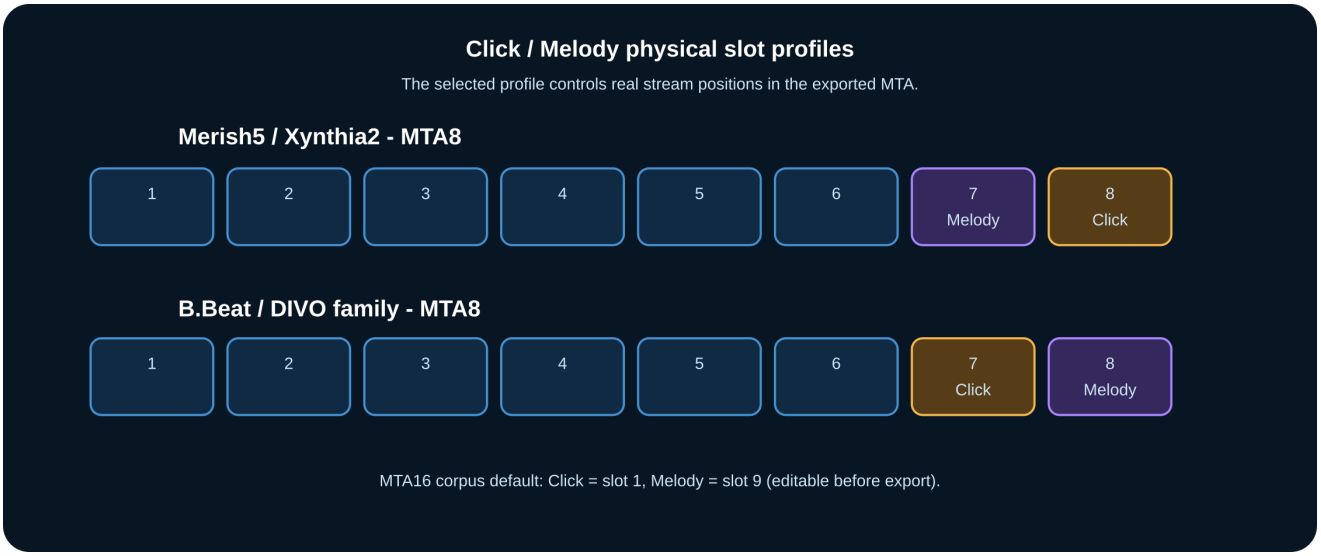


Figure 5 - MTA8/MTA16 export profiles.

MTA8 profiles preserve device-specific physical positions. The corpus-based MTA16 default uses Click 1 and Melody 9 and remains editable. Silent intermediate slots are generated when required to preserve the real stream number.

9. Operations and observability

Monitor health endpoints, persistent capacity, render duration, long jobs, application logs, FFmpeg/Demucs failures and service availability. Kubernetes probes should remain lightweight and independent from audio jobs.

Signal	Check
Health	`/api/health`
Storage	Free space and inodes
Jobs	Duration, state and server-side errors
Authentication	Login/OAuth/SMTP failures
Media	FFmpeg, Demucs, waveform/export logs

10. Upgrade and rollback

Before upgrading, create a backup, review release notes and production-gate results, update images/manifests and validate opening existing projects. For rollback, restore the previous application version and, when data compatibility requires it, the matching persistent-volume snapshot.

11. CI/CD and releases

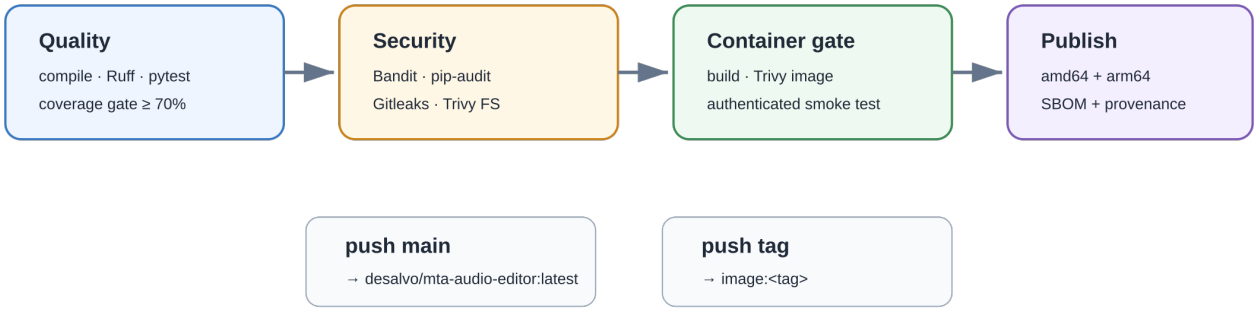


Figure 6 - Quality gates and packaging.

The production gate includes linting, tests, security checks and packaging validation. Native/mobile artifacts use dedicated build steps. Signing credentials belong in CI Secrets and must not be committed.

Release identity: `VERSION` stores the public product version (for example `0.2.0`), `REVISION` identifies the exact source/package revision (for example 154), and `BUILD_INFO` remains the independent 14-digit build identifier. Stable tags use `v0.2.0`; artifacts may include `-r154`. Platform-required *versionCode*/*CFBundleVersion* values are technical packaging codes and do not replace `BUILD_INFO`.

12. Administrator troubleshooting

Symptom	Check
413 / upload rejected	Align application and proxy/Ingress upload limits
Stem job is very slow	CPU/GPU, RAM, temporary space and Demucs model
Login/email failure	SMTP, public URL, authentication logs and user status
Pod cannot write to PVC	Storage permissions, UID/GID and fsGroup
Mobile custom server unavailable	Custom URL, TLS, DNS and reachability from the device
MTA rejected by hardware	Slot profile, target capacity, firmware and target-device validation

On-device Demucs for iPhone/iPad - Core ML provisioning

The server can distribute Core ML-compatible Demucs models by setting `MTA_DEMUCS_COREML_MODEL_DIR` to a directory containing `demucs-2.mlmodel`, `demucs-4.mlmodel`, `demucs-6.mlmodel`, and/or `demucs-8.mlmodel`. The iOS/iPadOS client downloads models using the authenticated session, compiles them locally, and caches them in Application Support. Later separations can run offline on the device; Auto mode keeps the server as fallback for missing models, material over the local safety limit, or inference errors.

Native updates and Demucs models

Native Windows, macOS, Android, iPhone and iPad applications check GitHub for newer versions. The **Stable** channel uses tagged releases only, while **Early release** also includes the rolling `early-main` prerelease produced by the latest successful main build.

On iPhone/iPad the app automatically provisions a default 4-stem Demucs Core ML model: it prefers a bundled validated model and otherwise downloads it from the configured server. SHA-256 fingerprints are compared with the server inventory and installed models are refreshed when the fingerprint changes. Once installed, the local model remains available offline.

On iOS/iPadOS application updates must still use TestFlight/App Store or another operating-system-authorized distribution mechanism.

Automatic mobile Demucs model updates

The Docker/Kubernetes server periodically refreshes Core ML (iPhone/iPad) and ONNX (Android) model repositories from an HTTPS manifest, verifies SHA-256 and replaces files atomically. Mobile clients check the catalogue every six hours and at startup. **Update on Wi-Fi only** is enabled by default, so periodic model updates download only over Wi-Fi unless the switch is explicitly disabled. The baseline 4-stem model is provisioned at the first available network opportunity.

Extended stem model registry

Configure additional server models through `MTA_DEMUCS_MODEL_REGISTRY` or `MTA_DEMUCS_MODEL_REGISTRY_FILE`. Each profile declares `model`, `stem_count`, `stem_labels`, and optionally `engine`/`display_name`. Core ML/ONNX repositories are discovered dynamically from `demucs-N` files, up to 64 stems.

Dynamic Demucs model management

The server periodically refreshes the full configured model catalogue, including official Demucs models available in the runtime and Core ML/ONNX artifacts published by the manifest. Mobile and desktop clients download the selected model on demand when splitting is requested. Local options can delete, re-download, or force-update one or more models. Administrators can blacklist a model: managed server artifacts are deleted and the model disappears from the web/client catalogue until it is removed from the blacklist.

Runtime model tokens

`POST /api/models/token` issues revocable Bearer tokens limited to `models:read`. Core ML, ONNX and native bundle downloads require an authenticated session or such a token. CI uses assets from the GitHub `demucs-models` release rather than runtime credentials.

Lyrics/Chords engines and MP4

For maximum lyrics accuracy the default is `MTA_LYRICS_WHISPER_MODEL=large-v3`; `MTA_LYRICS_BEAM_SIZE` is also configurable; at temperature 0 a deterministic beam search with increased patience is used. For chords, Sonic Annotator with the Chordino/NNLS-Chroma plugin is preferred automatically when installed. MP4 export requires FFmpeg with the `ass`/libass filter and H.264/AAC codecs.

SIAE / Soundreef repertoire integration

SIAE and Soundreef are enabled by default in repertoire searches. Their official public pages always remain available as a fallback. For structured server-to-server lookup, configure authorized/licensed HTTPS endpoints through `MTA_RIGHTS_SIAE_SEARCH_URL` and/or `MTA_RIGHTS_SOUNDREEF_SEARCH_URL`. The endpoint must return JSON with a list under `results`, `items`, `works` or `data`; common fields such as title/original_title, authors/writers/composers, performers/artists, publishers/editors, ISWC, IPI and work_id are recognized. The application does not assume private or undocumented collecting-society endpoints.